

2.1 Tipado Fuertemente Estático y Sistema de Tipos en Dart

En lenguajes interpretados o de tipado débil (como JavaScript o PHP tradicional), los errores de tipo explotan en la cara del usuario final en tiempo de ejecución. En Dart, el compilador es nuestro primer filtro de calidad.

El sistema de tipos de Dart es **Sound** (sólido). Esto garantiza dos cosas fundamentales:

1. Una variable no puede contener un valor que no corresponda a su tipo asignado.
2. No existen sorpresas en tiempo de ejecución referentes a tipos: si el compilador determina que una variable es de tipo **String**, en runtime *siempre* será un **String**.

```
// Esto provocará un error en tiempo de compilación directamente en VS Code:  
String nombreApp = "ESCOM Mobile";  
nombreApp = 42; // Error: A value of type 'int' can't be assigned to a variable of  
type 'String'.
```

Tipado Explícito vs. Inferencia de Tipos (**var**)

No siempre es necesario escribir el tipo de dato de forma explícita. Dart cuenta con **Type Inference** (inferencia de tipos). Cuando declaramos una variable con **var**, el compilador analiza el valor asignado a la derecha e infiere el tipo automáticamente:

```
var puertoServidor = 8080; // Dart infiere automáticamente que es 'int'  
// puertoServidor = "8080"; // ERROR: Sigue siendo 'int', no puedes cambiar su  
tipo.
```

2.2 Mutabilidad e Inmutabilidad: **var**, **final** y **const**

Saber cuándo una variable puede mutar y cuándo debe permanecer inmutable en memoria es crucial para optimizar el rendimiento de nuestras aplicaciones móviles y evitar efectos secundarios inesperados (*side effects*).

Palabras clave	Asignación de Valor	Momento de Evaluación	Uso Recomendado en Flutter / Mobile
var	Reassignable (mutable)	Tiempo de ejecución (<i>Runtime</i>)	Variables de contadores, estados cambiantes locales.
final	Única asignación (immutable)	Tiempo de ejecución (<i>Runtime</i>)	Respuestas de APIs, datos del usuario, IDs generados.
const	Única asignación (immutable)	Tiempo de compilación (<i>Compile-time</i>)	Constantes matemáticas, URLs base, Widgets estáticos.

Nota de arquitectura: Usar `const` le permite al compilador de Dart almacenar el valor en un espacio de memoria canónico reutilizable. En Flutter, declarar `const` en los `Widgets` estáticos evita re-renderizados innecesarios en la GPU, elevando los cuadros por segundo (FPS) de la app.

2.3 Tipos de Datos Nativos y Colecciones Nativas

Dart provee un conjunto rico de tipos primarios donde **todo es un objeto** (incluso los números hereda de la clase `Object`).

Números (`num`, `int`, `double`)

- `int`: Valores enteros de 64 bits (dependiendo de la plataforma target).
- `double`: Números de punto flotante de 64 bits bajo el estándar IEEE 754.
- `num`: Clase abstracta padre de `int` y `double`. Útil cuando una función acepta ambos.

Cadenas de Texto (`String`)

Las cadenas en Dart son secuencias UTF-16. Permiten **interpolación de variables** limpia usando `$` o `${}` para expresiones:

```
int version = 3;
String info = 'Estamos usando Dart SDK ${version + 0.5}';
```

Buleanos (`bool`)

Representan valores de verdad rigurosos: `true` y `false`. A diferencia de C++ o JS, en Dart las condiciones de estructuras `if` **exigen** explícitamente un tipo `bool` (no existen valores "truthy" o "falsy" como considerar `1` o `0` como booleanos).

Dynamic y Object: ¿Cuándo usarlos?

- `Object`: Es la raíz del árbol de jerarquía de clases. Si declaras `Object valor = "Hola"`, el compilador sabe que es un objeto, pero solo te permitirá invocar métodos propios de la clase `Object` (`toString()`, `hashCode`), a menos que hagas un casting explícito (`as String`).
- `dynamic`: Desactiva la verificación estática de tipos. Puedes invocar cualquier método o propiedad sin que el compilador proteste. **Peligro:** Si la propiedad no existe, tu app colapsará en runtime. *Evita su uso salvo en la deserialización inicial de JSONs heterogéneos.*

2.4 Operadores Avanzados en Dart

Además de los operadores aritméticos (+, -, *, /), relacionales (==, !=, >, <) y lógicos (&&, ||), Dart nos ofrece operadores optimizados para simplificar código:

1. **División Entera (~/)**: Retorna únicamente la parte entera del cociente.

```
int resultado = 10 ~/ 3; // Resultado: 3
```

2. **Operador de Cascada (`..` y `?..`):** Permite realizar una secuencia de operaciones sobre las propiedades o métodos de un mismo objeto sin repetir el nombre de la variable.

3. **Operadores Null-Aware:**

- `??` (Null-Coalescing): Devuelve el valor de la izquierda si no es nulo; si lo es, devuelve el de la derecha.
- `??=` (Asignación si es nulo): Asigna un valor solo si la variable actualmente contiene `null`.
- `?.` (Acceso condicional seguro): Ejecuta la propiedad/método solo si la variable no es nula.

2.5 Ejemplo

Creen el archivo `bin/d2_variables_operadores.dart` en VS Code. El siguiente programa simula la ingesta, parsing de cadenas, cálculo numérico y validaciones con seguridad nula para la inscripción de asignaturas en ESCOM.

```
/// Módulo D2: Variables, Tipos de Datos y Operadores
/// Demostración de tipado estático, constantes, conversiones y operadores
avanzados.

void main() {
  print('=== MÓDULO D2: SISTEMA DE PROCESAMIENTO DE DATOS - ESCOM ===\n');

  // 1. Uso de const vs final
  const String codigoUnidadAprendizaje = 'DAMN-7S'; // Constante en compilación
  final DateTime fechaEjecucion = DateTime.now(); // Determinado en tiempo de
ejecución

  print('Código del curso: $codigoUnidadAprendizaje');
  print('Fecha de procesamiento: $fechaEjecucion\n');

  // 2. Manejo de Números y Operaciones Aritméticas Avanzadas
  final String entradaCalificaciones = '85,90,78,95,60'; // Cadena tipo CSV
  final int totalEstudiantes = 5;

  // Transformación y Parsing (String -> List<int>)
  final List<int> listaNotas = entradaCalificaciones
    .split(',')
    .map((notaStr) => int.parse(notaStr))
    .toList();

  print('Calificaciones procesadas: $listaNotas');

  // Sumatoria usando fold (Operador de acumulación)
  final int sumaTotal = listaNotas.fold(0, (acumulado, nota) => acumulado + nota);

  // Operadores de división
```

```

    final double promedioExacto = sumaTotal / totalEstudiantes; // División decimal
    (//)
    final int promedioEntero = sumaTotal ~/ totalEstudiantes; // División entera
    (~//)
    final int residuo = sumaTotal % totalEstudiantes; // Módulo (%)

    print('Suma total de puntos: $sumaTotal');
    print('Promedio exacto: $promedioExacto');
    print('Promedio truncado (entero): $promedioEntero');
    print('Residuo del cálculo: $residuo\n');

    // 3. Demostración de Operadores Null-Aware (??, ??=, ?.)
    String? tokenSesion; // Inicialmente nulo

    // Asignación condicionada con ??=
    tokenSesion ??= 'TOKEN_TEMPORAL_ESCOM_2026';
    print('Token de sesión asignado: $tokenSesion');

    // Re-asignación intentada con ??= (No cambiará porque ya no es nulo)
    tokenSesion ??= 'TOKEN_NUEVO_IGNORADO';
    print('Token tras intento de reasignación: $tokenSesion\n');

    // Evaluación con operador de coalescencia de nulos (??)
    String? usuarioInput;
    final String nombreMostrar = usuarioInput ?? 'Alumno Invitado';
    print('Bienvenido: $nombreMostrar\n');

    // 4. Demostración del Operador de Cascada (..)
    print('--- Generación de Reporte con Operador de Cascada ---');
    final StringBuffer bufferReporte = StringBuffer()
        ..write('REPORTE FINAL:\n')
        ..write(' - Asignatura: $codigoUnidadAprendizaje\n')
        ..write(' - Promedio General: ${promedioExacto.toStringAsFixed(2)}\n')
        ..write(' - Estado de Aprobación: ')
        ..write(promedioExacto >= 70 ? 'GRUPO APROBADO' : 'GRUPO EN RIESGO');

    print(bufferReporte.toString());
    print('\n=====');
}

```

Instrucciones de Ejecución en VS Code:

```
dart run bin/d2_variables_operadores.dart
```

2.6 Ejercicios Prácticos

Para afianzar los conceptos de conversión de tipos, mutabilidad y operadores avanzados, implementen los siguientes dos programas en su entorno local:

Ejercicio 1: Convertidor de Formatos de Red y Subneteo Simplificado

Crea un script en Dart que realice conversiones numéricas y operaciones a nivel de datos:

1. Define una dirección IP como una cadena (`String ip = "192.168.1.50"`).
2. Separa los octetos de la IP utilizando `.split('.')` y convierte cada valor a entero (`int`).
3. Calcula la **Dirección de Red** equivalente si la máscara de red es un valor estático `255.255.255.0`, aplicando el operador relacional de nivel de bits AND (`&`) a cada octeto.
4. Muestra en consola la IP original, la máscara utilizada y la IP de red calculada.

Ejercicio 2: Gestor de Perfiles con Nulabilidad y Operadores Null-Aware

Crea un sistema simulado de actualización de perfil de usuario:

1. Declara variables nulas para un perfil de alumno: `String? telefono`, `String? correoAlternativo`, `int? edad`.
 2. Utiliza el operador `??=` para asignar valores por defecto solo a aquellas variables que sigan siendo nulas.
 3. Imprime una ficha de usuario en consola asegurando que si `correoAlternativo` es nulo, muestre `"NO REGISTRADO"`, utilizando el operador `??`.
 4. Utiliza el operador de cascada (`..`) con un objeto de tipo `StringBuffer` para estructurar el mensaje final antes de imprimirlo.
-